

3. 关系模式分解

如果某关系模式存在修改异常等问题，则可通过分解该关系模式来解决问题。将一个关系模式分解成几个子关系模式，需要考虑的是该分解是否保持函数依赖，是否是无损联接。

无损联接分解的形式定义如下：设 R 是一个关系模式， F 是 R 上的一个函数依赖集。 R 分解成数据库模式 $\delta = \{R_1, \dots, R_k\}$ 。如果对 R 中每个满足 F 的关系 r 都有下式成立：

$$r = \pi_{R_1}(r) \bowtie \pi_{R_2}(r) \bowtie \dots \bowtie \pi_{R_k}(r)$$

则称分解 δ 相对于 F 是无损联接分解，否则称为损失联接分解。

要根据上述定义来判断一个分解是否是无损联接，这是一件很困难的事情，下面是一个很有用的无损联接分解判定定理：设 $\rho = \{R_1, R_2\}$ 是 R 的一个分解， F 是 R 上的函数依赖集，那么分解 ρ 相对于 F 是无损联接分解的充要条件是 $(R_1 \cap R_2) \rightarrow (R_1 - R_2)$ 或 $(R_1 \cap R_2) \rightarrow (R_2 - R_1)$ 。要注意的是，这两个条件只要有任意一个条件成立就可以了。

设数据库模式 $\delta = \{R_1, \dots, R_k\}$ 是关系模式 R 的一个分解， F 是 R 上的函数依赖集， δ 中每个模式 R_i 上的函数依赖集是 F_i 。如果 $\{F_1, F_2, \dots, F_k\}$ 与 F 是等价的（即相互逻辑蕴涵），则称分解 δ 保持函数依赖。如果分解不能保持函数依赖，则 δ 的实例上的值就可能有违反函数依赖的现象。

5.3 数据库控制功能

要想使数据库中的数据达到应用的要求，必须对其进行各种控制，这就是 DBMS 的控制功能，包括并发控制、性能优化、数据库的完整性和安全性，以及数据备份与恢复等。这些技术虽然给人们的感觉是边缘性技术，但对 DBMS 的应用而言却是至关重要的。

5.3.1 并发控制

在多用户共享系统中，许多事务可能同时对同一数据进行操作，称为并发操作。此时，DBMS 的并发控制子系统负责协调并发事务的执行，保证数据库的完整性不受破坏，同时，避免用户得到不正确的数据。

1. 事务的基本概念

DBMS 运行的基本工作单位是事务，事务是用户定义的一个数据库操作序列，这些操作序列要么全做，要么全不做，是一个不可分割的工作单位。事务具有以下特性：

(1) 原子性 (Atomicity)。事务是数据库的逻辑工作单位，事务的原子性保证事务包含的一组更新操作是原子不可分的，也就是说，这些操作是一个整体，不能部分地完成。

(2) 一致性 (Consistency)。一致性是指使数据库从一个一致性状态变到另一个一致性状态。例如，在转账的操作中，各账户金额必须平衡。一致性与原子性是密切相关的，一致性在逻辑上不是独立的，它由事务的隔离性来表示。

(3) 隔离性 (Isolation)。隔离性是指一个事务的执行不能被其他事务干扰，即一个事务内

部的操作及使用的数据对并发的其他事务是隔离的，并发执行的各个事务之间不能互相干扰。它要求即使有多个事务并发执行，但看上去每个事务是按串行调度执行一样。这一性质也称为可串行性，也就是说，系统允许的任何交错操作调度等价于一个串行调度。

(4) 持久性 (Durability)。持久性也称为永久性，是指事务一旦提交，改变就是永久性的，无论发生何种故障，都不应该对其有任何影响。

事务的原子性、一致性、隔离性和持久性通常统称为 ACID 特性。

2. 数据不一致问题

数据库的并发操作会带来一些数据不一致问题，例如，丢失修改、读“脏数据”和不可重复读等。

(1) 丢失修改。事务 A 与事务 B 从数据库中读入同一数据并修改，事务 B 的提交结果破坏了事务 A 提交的结果，导致事务 A 的修改被丢失。例如，有 T1、T2 两个事务，其执行顺序如表 5-7 所示，则“③ A=A-5，写回”操作会被“④ A=A-8，写回”操作覆盖掉，“③ A=A-5，写回”将不起任何作用。

表 5-7 丢失修改的实例

T1	T2
①读 A=10	
②	读 A=10
③ A=A-5，写回	
④	A=A-8，写回

(2) 读“脏数据”。事务 A 修改某一数据，并将其写回磁盘，事务 B 读取同一数据后，事务 A 由于某种原因被撤销，这时事务 A 已修改过的数据恢复原值，事务 B 读到的数据就与数据库中的数据不一致，是不正确的数据，称为“脏数据”。例如，有 T1、T2 两个事务，其执行顺序如表 5-8 所示，则 T2 中“读 A=70”就是读的“脏数据”。

表 5-8 读“脏数据”的实例

T1	T2
①读 A=20 A=A+50 写回 70	
②	
③ ROLLBACK A 恢复为 20	读 A=70

(3) 不可重复读。不可重复读是指事务 A 读取数据后，事务 B 执行了更新操作，事务 A 使用的仍是更新前的值，造成了数据不一致性。例如，有 T1、T2 两个事务，其执行顺序如表 5-9 所示。

表 5-9 不可重复读的实例

T1	T2
①读 A=20 读 B=30 求和 =50	
②	读 A=20 A=A+50 写 A=70
③读 A=70 读 B=30 求和 =100 (验算不对)	

在表 5-9 中, T1 事务为了确保其重要计算无误, 所以采用了验算的方式, 两次独立取出数据并运算, 最后进行验算 (即比较两次运算结果是否相同)。在此处, 虽然两次计算都没错, 但由于在两次操作之间的时间间隔中, T2 对数据进行了修改, 导致验算结果不正确, 这就是不可重复读问题。

3. 封锁协议

处理并发控制的主要方法是采用封锁技术, 主要有两种封锁, 分别是 X 封锁和 S 封锁。

(1) 排他型封锁 (X 封锁)。如果事务 T 对数据对象 A (可以是数据项、元组和数据集, 以至整个数据库) 实现了 X 封锁, 那么只允许事务 T 读取和修改数据 A, 其他事务要等事务 T 解除 X 封锁以后, 才能对数据 A 实现任何类型的封锁。可见, X 封锁只允许一个事务独锁某个数据, 具有排他性。

(2) 共享型封锁 (S 封锁)。X 封锁只允许一个事务独锁和使用数据, 要求太严, 需要适当从宽。例如, 可以允许并发读, 但不允许修改, 这就产生了 S 封锁的概念。S 封锁的含义是, 如果事务 T 对数据 A 实现了 S 封锁, 那么允许事务 T 读取数据 A, 但不能修改数据 A, 在所有 S 封锁解除之前, 决不允许任何事务对数据 A 实现 X 封锁。

在多个事务并发执行的系统中, 主要采取封锁协议来进行处理。常见的封锁协议如下:

(1) 一级封锁协议。事务 T 在修改数据 R 之前必须先对其加 X 锁, 直到事务结束才释放。一级封锁协议可防止丢失修改, 并保证事务 T 是可恢复的, 但不能保证可重复读和不读“脏数据”。

(2) 二级封锁协议。一级封锁协议加上事务 T 在读取数据 R 之前先对其加 S 锁, 读完后即可释放 S 锁。二级封锁协议可防止丢失修改, 还可防止读“脏数据”, 但不能保证可重复读。

(3) 三级封锁协议。一级封锁协议加上事务 T 在读取数据 R 之前先对其加 S 锁, 直到事务结束才释放。三级封锁协议可防止丢失修改、读“脏数据”, 且能保证可重复读。

(4) 两段锁协议。所有事务必须分两个阶段对数据项加锁和解锁。其中, 扩展阶段是在对任何数据进行读、写操作之前, 首先要申请并获得对该数据的封锁; 收缩阶段是在释放一个封锁之后, 事务不能再申请和获得任何其他封锁。若并发执行的所有事务均遵守两段锁协议, 则